1. Install Git on Amazon Linux

Step 1: Connect to Your Instance

Connect to your Amazon Linux instance using SSH.

Step 2: Install Git

Command:

```
sudo yum install git -y
```

Command explanation:

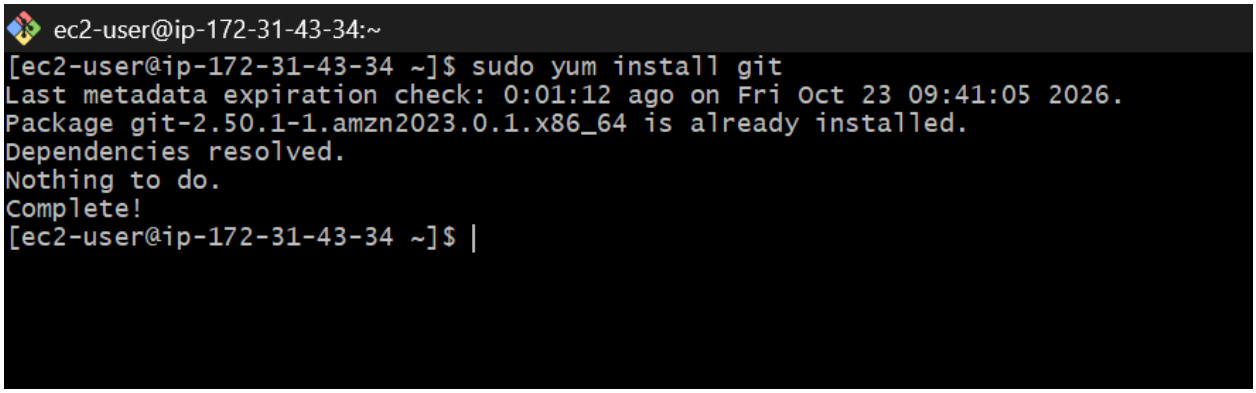
- `sudo` — Runs the command with administrator privileges.
- `yum install git` — Installs Git.
- `-y` — Automatically confirms the installation.

Step 3: Verify the Installation

Command:

```
git --version
```

Command explanation: Displays the installed Git version.



2. Create a repo in github with README.md and .ignore file.

Step 1: Open GitHub

Go to https://github.com and sign in.

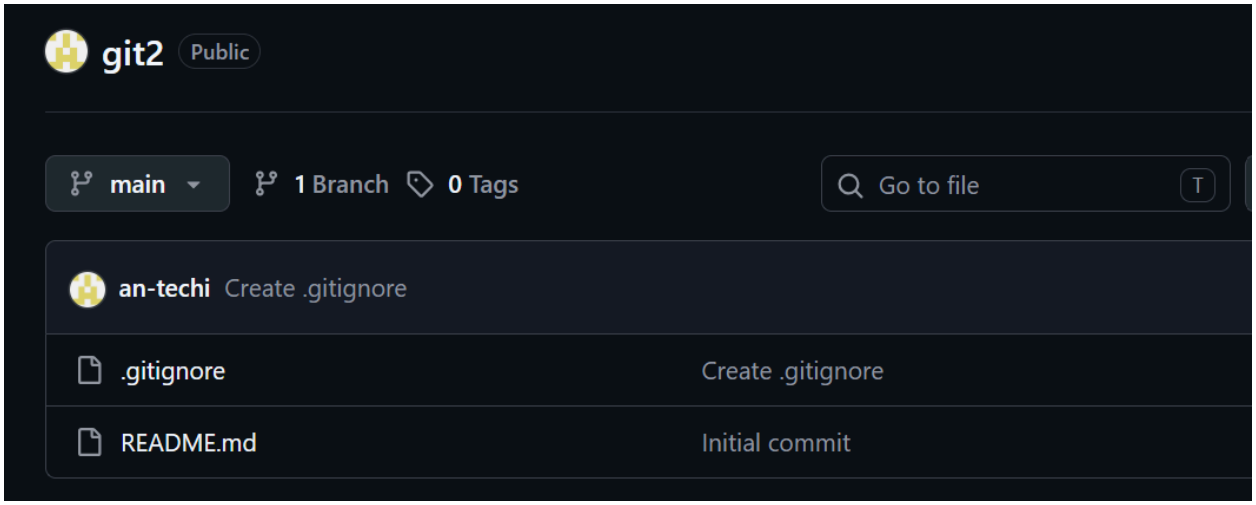
Step 2: Create a Repository

1. Click the **+** icon and select **New repository**.
2. Enter a repository name.
3. Add a description (optional).
4. Select Public or Private.
5. Check **Add a README file**.
6. Click **Create repository**.

Step 3: Create the `.gitignore` File

1. Open your repository.
2. Click **Add file** → **Create new file**.
3. Enter `.gitignore` as the filename.
4. Add files or patterns you want Git to ignore.
5. Click **Commit changes**.

Explanation: `README.md` describes your project, while `.gitignore` specifies files Git should not track.



3. Clone the created repo to local.

Step 1: Copy the Repository URL

1. Open your repository on GitHub.
2. Click **Code**.
3. Copy the HTTPS URL.

Step 2: Clone the Repository

Open your terminal and run:

```
git clone <repository-URL>
```

Command explanation:

- `git clone` — Downloads a copy of the repository to your local machine.
- `<repository-URL>` — Replace this with your copied GitHub URL.

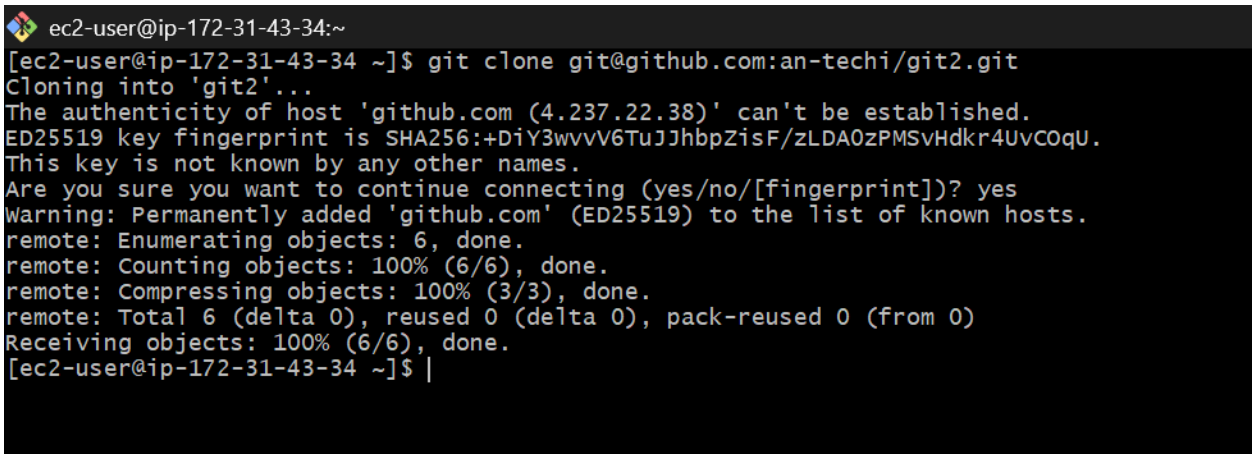
Step 3: Navigate to the Repository

Run:

```
cd <repository-name>
```

Command explanation:

- `cd` — Changes the current directory to the cloned repository.



4. create two files in local repo

Step 1: Create Two Files

Run these commands inside your local repository:

echo "hello world!" > test1.txt

echo "hey there!" > test2.txt

Command explanation:

- `echo` — Prints the given text.
- `>` — Writes the text into a file, creating it if it doesn't exist.
- `test1.txt` and `test2.txt` — The files created with the specified content.

Step 2: Verify the Files

ls

Command explanation: Lists the files in your current directory.



5. Commit two files and push to central Repository.

Step 1: Check File Status

git status

Command explanation: Shows which files are new, modified, or ready to commit.

Step 2: Add the Two Files

git add test1.txt test2.txt

Command explanation: Stages both files for the next commit.

Step 3: Commit the Files

git commit -m "Add two text files"

Command explanation:

- `git commit` — Saves the staged changes in Git history.
- `-m` — Adds a message describing the commit.

Step 4: Push to the Central Repository

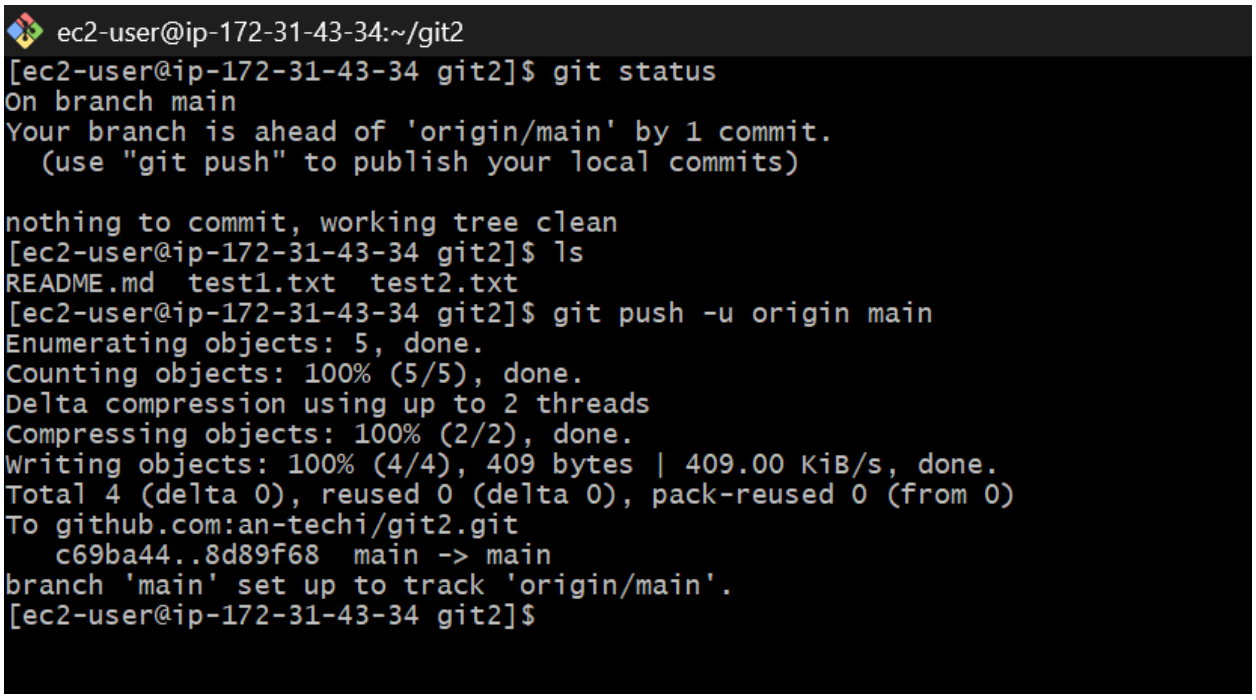
git push

Command explanation: Uploads your committed changes to GitHub.

If Git asks you to specify the remote branch, run:

git push -u origin main

Explanation: `origin` is the usual name for the GitHub remote, and `main` is the branch name. Use your actual branch name if it differs.



6. Create a branch in local and create a sample file and push to central.

Step 1: Create a New Branch

git checkout -b develop

Command explanation:

- `git checkout` — Switches branches.
- `-b develop` — Creates a new branch named `develop` and switches to it.

Step 2: Create a Sample File

echo "This is a sample file" > samplefile.txt

Command explanation:

- `echo` — Prints the given text.
- `>` — Writes the text into a file, creating it if it doesn't exist.
- `samplefile.txt` — The file being created.

Step 3: Add the File

git add .

Command explanation: Stages the new file for committing.

Step 4: Commit the File

git commit -m "added sample file in develop branch"

Command explanation:

- `git commit` — Saves the staged changes in Git history.
- `-m` — Adds a commit message.

Step 5: Push the Branch to GitHub

git push -u origin develop

Command explanation:

- `git push` — Uploads your commit to GitHub.
- `-u` — Sets the upstream branch for future pushes.
- `origin` — The remote repository name.
- `develop` — The branch being pushed.

```
[ec2-user@ip-172-31-43-34 git2]$ git checkout develop
Switched to branch 'develop'
[ec2-user@ip-172-31-43-34 git2]$ echo "This is a sample file " > samplefile.txt
[ec2-user@ip-172-31-43-34 git2]$ git status
On branch develop
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        samplefile.txt

nothing added to commit but untracked files present (use "git add" to track)
[ec2-user@ip-172-31-43-34 git2]$ git add .
[ec2-user@ip-172-31-43-34 git2]$ git commit "added sample file in develop branch"
error: pathspec 'added sample file in develop branch' did not match any file(s) known to git
[ec2-user@ip-172-31-43-34 git2]$ git commit -m "added sample file in develop branch"
[develop bd436cd] added sample file in develop branch
 1 file changed, 1 insertion(+)
 create mode 100644 samplefile.txt
[ec2-user@ip-172-31-43-34 git2]$ git push
fatal: The current branch develop has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin develop

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

[ec2-user@ip-172-31-43-34 git2]$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
[ec2-user@ip-172-31-43-34 git2]$ git status
On branch develop
nothing to commit, working tree clean
[ec2-user@ip-172-31-43-34 git2]$ git push -u origin develop
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 309 bytes | 309.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
```

7. Create a branch in github and clone that to local.

Step 1: Create a Branch on GitHub

1. Open your repository: https://github.com/an-techi/git2
2. Click the branch selector.
3. Enter `feature`.
4. Click **Create branch: feature**.

Step 2: Clone the Feature Branch to Local

Run this command in your Amazon Linux terminal:

git clone -b feature git@github.com:an-techi/git2.git

Command explanation:

- `git clone` — Downloads the repository to your local machine.
- `-b feature` — Checks out the `feature` branch after cloning.
- `git@github.com:an-techi/git2.git` — The SSH URL of your GitHub repository.

Step 3: Enter the Cloned Repository

cd git2

Command explanation: Moves into the cloned repository directory.

Step 4: Verify the Branch

git branch --list

Command explanation: Lists local branches. The * indicates the current branch.

```
[ec2-user@ip-172-31-43-34 git2]$ cd feature
[ec2-user@ip-172-31-43-34 feature]$ git clone -b feature git@github.com:an-techi/git2.git
Cloning into 'git2'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 15 (delta 3), reused 6 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), done.
Resolving deltas: 100% (3/3), done.
[ec2-user@ip-172-31-43-34 feature]$ git branch --list
* develop
  main
[ec2-user@ip-172-31-43-34 feature]$ ls
git2
[ec2-user@ip-172-31-43-34 feature]$ ls
git2
[ec2-user@ip-172-31-43-34 feature]$ cd git2
[ec2-user@ip-172-31-43-34 git2]$ ls
README.md  feature.txt  test1.txt  test2.txt
[ec2-user@ip-172-31-43-34 git2]$ git branch --list
* feature
[ec2-user@ip-172-31-43-34 git2]$
```

8. Merge feature branch into main branch

Step 1: Check the Current Branch

git branch --list

Command explanation: Lists local branches. The `*` indicates the current branch (`main`).

Step 2: Merge the Feature Branch

git merge feature

Command explanation:

- `git merge` — Combines changes from another branch into the current branch.
- `feature` — The branch whose changes are being merged into `main`.



9. Merge the created branch with master in github by sending a pull request.

Step 1: Push the Feature Branch to GitHub

git checkout feature

git push -u origin feature

Command explanation:

- `git checkout feature` **— Switches to the** `feature` **branch.**
- `git push -u origin feature` **— Uploads the branch to GitHub.**

Step 2: Open GitHub

1. **Go to https://github.com/an-techi/git2**
2. **Click the Pull requests tab.**
3. **Click New pull request.**

Step 3: Set the Branches

- **Base:** `master` **(the branch receiving the changes).**
- **Compare:** `feature` **(the branch containing your changes).**

Step 4: Create the Pull Request

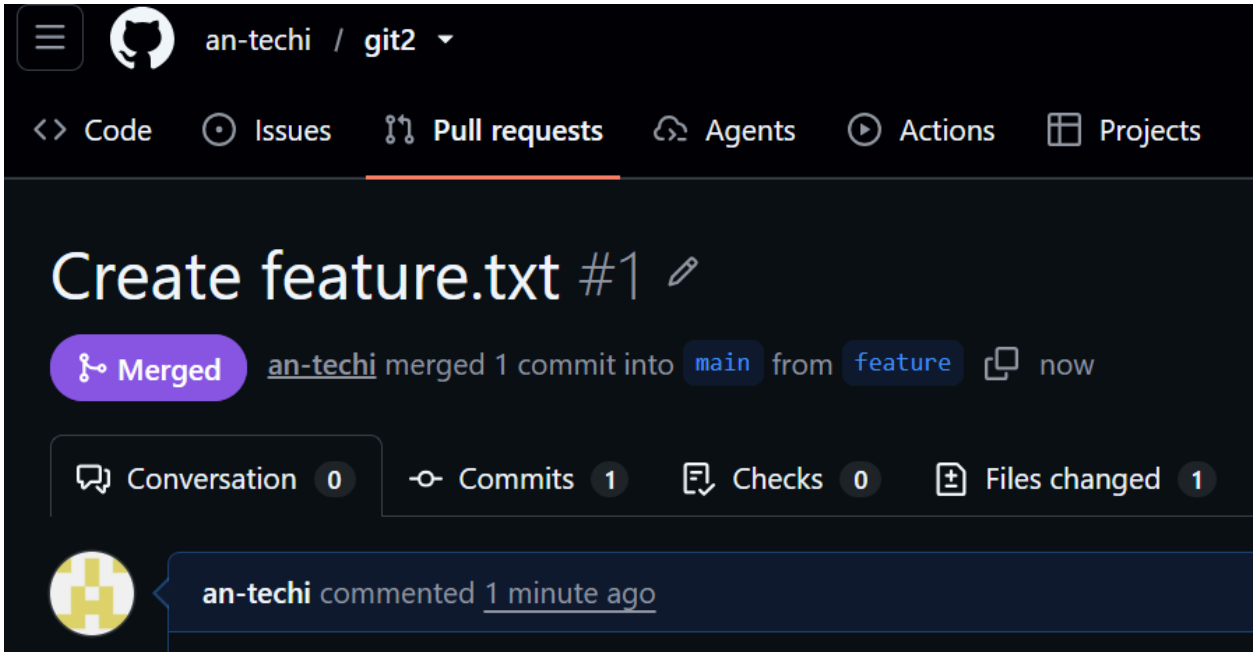
1. **Click Create pull request.**
2. **Enter a title, such as** `Merge feature into master`**.**
3. **Click Create pull request again.**

Step 5: Merge the Pull Request

1. **Open the pull request.**
2. **Click Merge pull request.**
3. **Click Confirm merge.**

Command explanation: A pull request lets you propose and review changes before merging one branch into another.

Note: If your repository uses `main` **instead of** `master`**, select** `main` **as the base branch.**



10. Create a file in local and send that to branch in github.

Step 1: Create a File

echo "This file is sent to GitHub" > send_to_git_hub.txt

Command explanation:

- **echo — Prints the given text.**
- **> — Writes the text into a file.**

Step 2: Add the File

git add send_to_git_hub.txt

Command explanation: Stages the file for committing.

Step 3: Commit the File

git commit -m "Add send_to_git_hub file"

Command explanation:

- `git commit` — Saves the staged changes.
- `-m` — Specifies the commit message.

Step 4: Pull Remote Changes Using Rebase

git pull --rebase origin main

Command explanation:

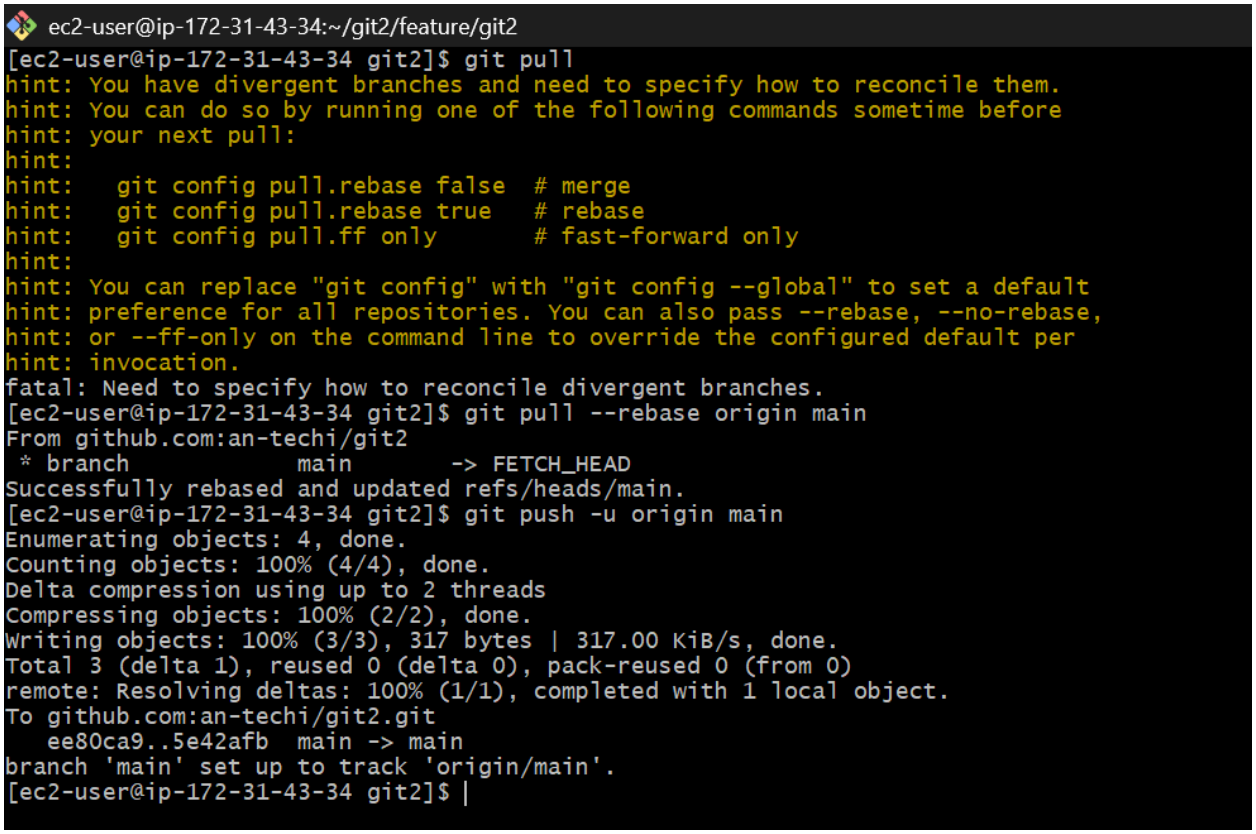
- `git pull` — Fetches and integrates remote changes.
- `--rebase` — Reapplies local commits on top of remote commits.
- `origin main` — Specifies the GitHub remote and `main` branch.

Step 5: Push the File to GitHub

git push -u origin main

Command explanation:

- `git push` — Uploads your commits to GitHub.
- `-u` — Sets the upstream branch for future pushes.
- `origin main` — Pushes to the `main` branch on GitHub.



11. Clone only a branch from github to local.

Step 1: Copy the SSH URL

Open your GitHub repository, click Code → SSH, and copy the repository URL.

Step 2: Clone Only the Required Branch

Run this command in your terminal:

git clone -b feature --single-branch git@github.com:an-techi/git2.git

Command explanation:

- `git clone` **— Downloads the repository.**
- `-b feature` **— Selects the** `feature` **branch.**
- `--single-branch` **— Downloads history only for the specified branch.**
- `git@github.com:an-techi/git2.git` **— Your repository's SSH URL.**

Step 3: Enter the Repository

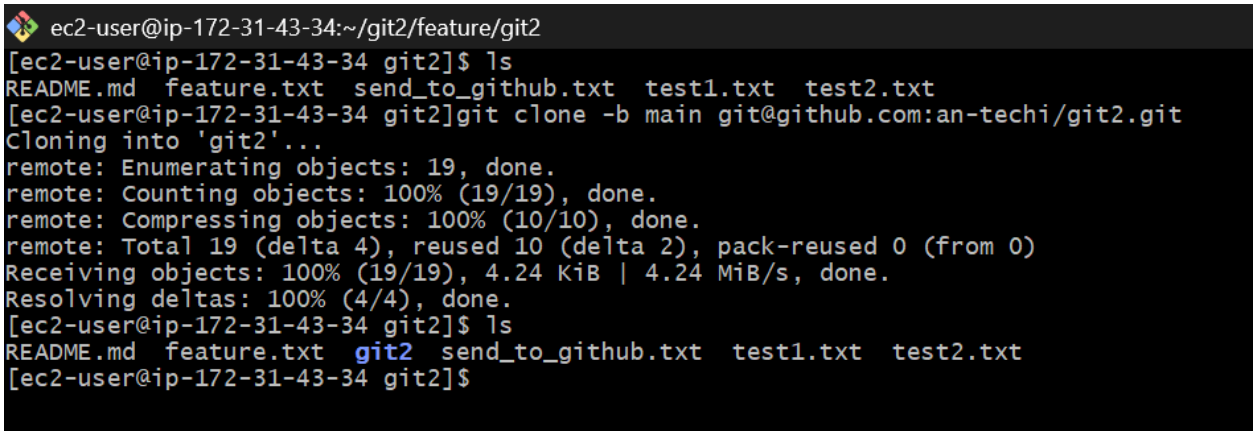
cd git2

Command explanation: Moves into the cloned repository directory.

Step 4: Verify the Branch

git branch

Command explanation: Displays the local branches. The `*` indicates the current branch, which should be `feature`.



12. Create a file with all passwords and make that untrackable with git.

Step 1: Create a Password File

touch passwords.txt

Command explanation: `touch` creates an empty file named `passwords.txt`. Do not store real passwords in a Git repository.

Step 2: Add the File to `.gitignore`

echo "passwords.txt" >> .gitignore

Command explanation:

- `echo` **— Outputs the filename.**
- `>>` **— Appends the filename to** `.gitignore` **without overwriting existing content.**
- `.gitignore` **— Tells Git which untracked files to ignore.**

Step 3: Verify the File Is Ignored

git status

Command explanation: Shows the repository's current changes. `passwords.txt` **should not appear as an untracked file.**

Step 4: Commit the `.gitignore` **Change**

git add .gitignore

git commit -m "Ignore password file"

Command explanation:

- `git add .gitignore` **— Stages the ignore rules.**
- `git commit -m` **— Saves the change with a message.**

Important: `.gitignore` **does not stop tracking a file that Git already tracks. If the file was previously committed, it needs to be removed from Git's index separately.**

```
ec2-user@ip-172-31-43-34:~/git2/feature/git2
[ec2-user@ip-172-31-43-34 git2]$ touch passwords.txt
[ec2-user@ip-172-31-43-34 git2]$ echo "passwords.txt" >> .gitignore
[ec2-user@ip-172-31-43-34 git2]$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        git2/

no changes added to commit (use "git add" and/or "git commit -a")
[ec2-user@ip-172-31-43-34 git2]$ git add .gitignore
git commit -m "Ignore password file"
[main ea83cce] Ignore password file
 1 file changed, 1 insertion(+)
[ec2-user@ip-172-31-43-34 git2]$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 296 bytes | 296.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:an-techi/git2.git
   5e42afb..ea83cce  main -> main
[ec2-user@ip-172-31-43-34 git2]$
```

13. Make a commit and make that commit reset without savings changes.

Step 1: Create a File

echo "Sample content" > sample.txt

Explanation: Creates `sample.txt` with sample text.

Step 2: Add and Commit the File

git add sample.txt

git commit -m "Add sample file"

Explanation:

- `git add` — Stages the file.
- `git commit -m` — Saves the staged changes in Git history.

Step 3: Reset the Commit Without Saving Changes

git reset --hard HEAD~1

Explanation:

- `git reset` — Moves the current branch to an earlier commit.
- `--hard` — Resets the staging area and working directory, discarding the changes.
- `HEAD~1` — Refers to the commit immediately before the latest commit.

This permanently discards the latest commit's changes from your working directory. Do not use it if you need to keep those changes.



14. Revert a commited commit to the older version.

Step 1: Check Commit History

git log --oneline

Command explanation: Displays the recent commits with their IDs and messages.

Step 2: Revert the Commit

git revert <commit-id>

Command explanation:

- `git revert` **— Creates a new commit that undoes the changes from an earlier commit.**
- `<commit-id>` **— Replace this with the ID of the commit you want to undo.**

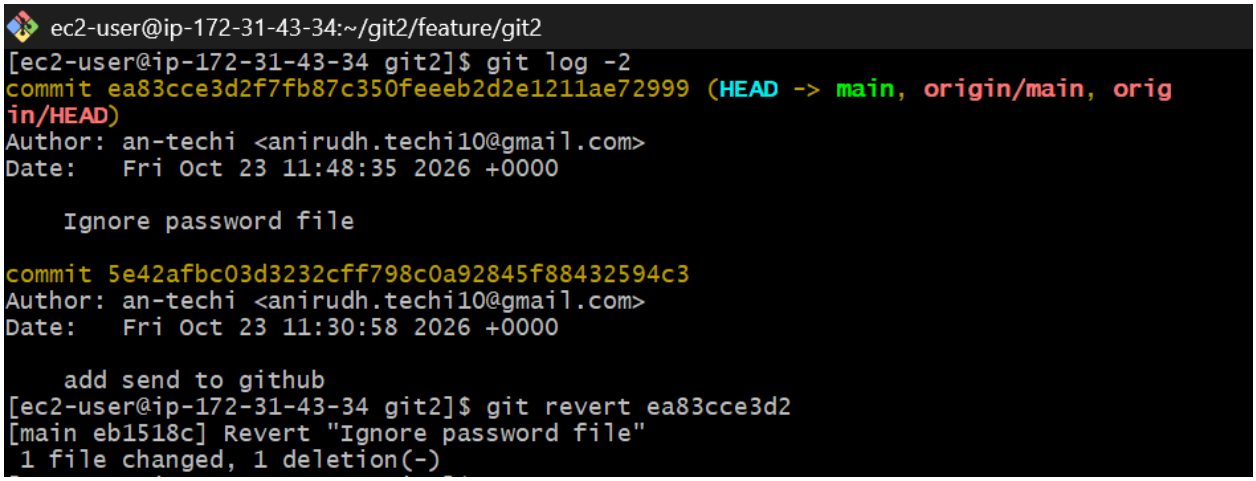
Step 3: Push the Revert to GitHub

git push origin main

Command explanation:

- `git push` **— Uploads the new revert commit to GitHub.**
- `origin main` **— Specifies the remote repository and the** `main` **branch.**

Unlike `git reset`, `git revert` **preserves the existing commit history.**



15. Push a file to stash without saving the changes and work on another file.

Step 1: Stash Current Changes

git stash -u

Command explanation:

- `git stash` **— Temporarily saves your uncommitted changes.**
- `-u` **— Includes untracked files.**

Step 2: Modify Another File

vi feature.txt

Command explanation: Opens `feature.txt` in the Vi editor so you can make changes.

- **Press `i` to start editing.**
- **Press `Esc` when finished.**
- **Type `:wq` and press Enter to save and exit.**

Step 3: Add and Commit the Changes

git add feature.txt

git commit -m "Update feature file"

Command explanation:

- `git add` **— Stages the file.**
- `git commit -m` **— Records the changes in Git history.**

Step 4: Push to GitHub

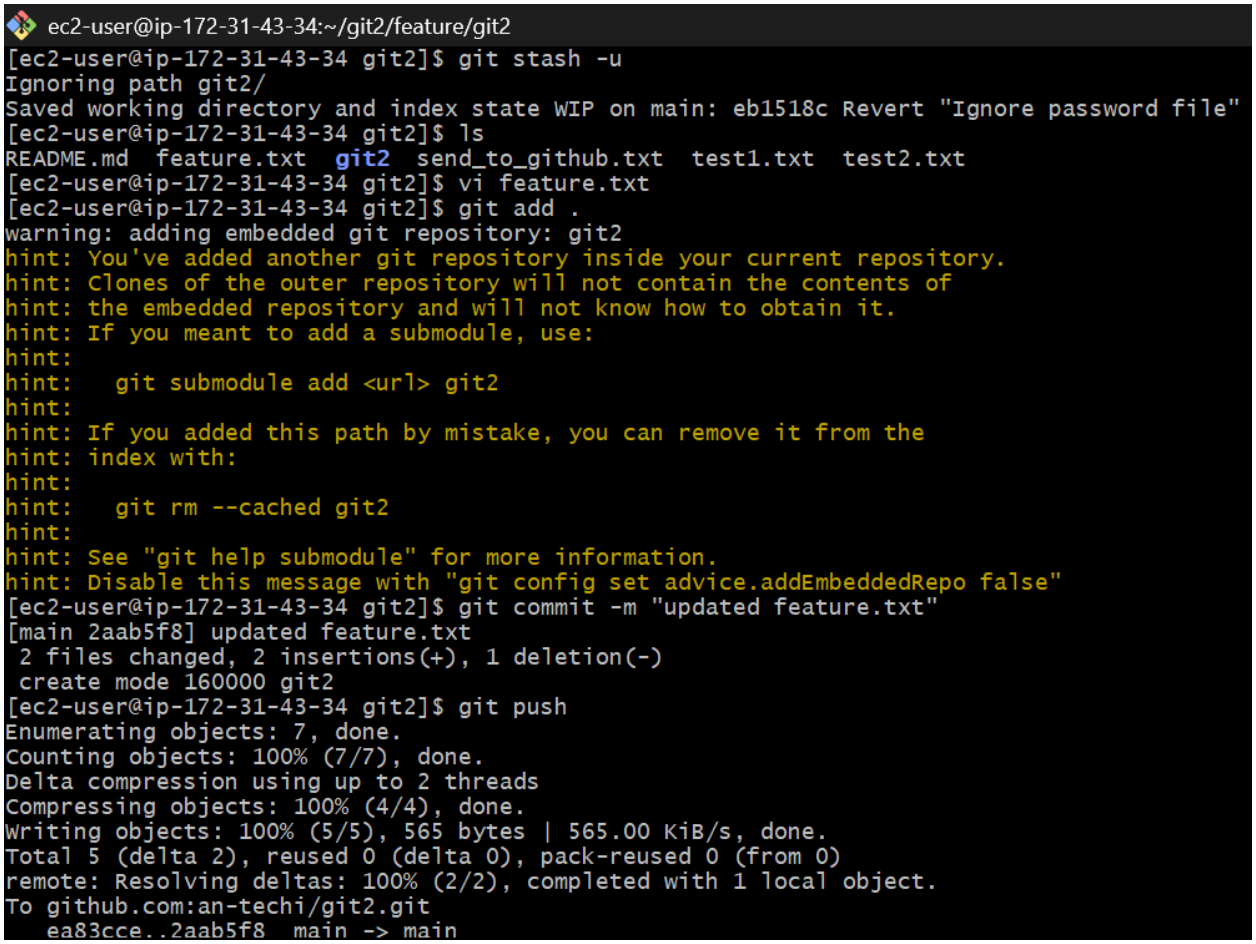
git push origin main

Command explanation: Uploads your commit to the remote `main` branch.

Step 5: Restore Stashed Changes (Optional)

git stash pop

Command explanation: Restores the stashed changes and removes that stash entry if successful.



16. Undo the stash file and start working on that again.

Step 1: Check Stash Status

git stash list

Command explanation: Lists saved stashes. Your output shows the stash was already popped and removed.

Step 2: Check Your Files

git status

Command explanation: Shows modified and untracked files. Your `passwords.txt` file is currently untracked.

Step 3: Start Working on the File

vi passwords.txt

Command explanation: Opens the file in the Vi editor so you can edit it.

- **Press `i` to start editing.**
- **Press `Esc` when finished.**
- **Type `:wq` and press Enter to save and exit.**

`git stash pop` has already restored the stash successfully. If you want to restore changes from the stash again, it has been removed from the stash list, so you would need to recover it from Git history if necessary.

```
[ec2-user@ip-172-31-43-34 git2]$ git stash pop
Already up to date.
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        passwords.txt

nothing added to commit but untracked files present (use "git add" to track)
Dropped refs/stash@{0} (8be9b8e431da46afa99b27ef0a3959d70c0ded7a)
```

17. Generate a ssh-keygen and configure into github.

Step 1: Generate an SSH Key

Run the command:

ssh-keygen

Command explanation: Generates a public and private SSH key pair. Press Enter to accept the default file location and follow the prompts.

Step 2: Copy the SSH Public Key

Display the public key:

cat ~/.ssh/id_rsa.pub

Command explanation: Displays the public key so you can copy it. If your generated key has a different filename, use that filename instead.

Step 3: Add the Key to GitHub

1. **Log in to GitHub.**
2. **Go to Settings → SSH and GPG keys.**
3. **Click New SSH key.**
4. **Enter a title.**
5. **Paste the copied public key.**
6. **Click Add SSH key.**

Step 4: Test the SSH Connection

ssh -T git@github.com

Command explanation: Tests whether your SSH key authenticates with GitHub.

Important: Copy only the public key (`.pub` file), never the private key.

```
[ec2-user@ip-172-31-45-114 git_test]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_rsa):
Enter passphrase for "/home/ec2-user/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/id_rsa
Your public key has been saved in /home/ec2-user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:s/bf9jV8qbX2z6CaQO+l77u+OK7+qUBn1IxL8lFj/vU ec2-user@ip-172-31-45-114.ap-southeast-2.compute.internal
The key's randomart image is:
```

18. Configure webhooks to github.

Step 1: Open Your GitHub Repository

Open your repository on GitHub and go to Settings.

Step 2: Open Webhooks

1. **Click Webhooks.**
2. **Click Add webhook.**

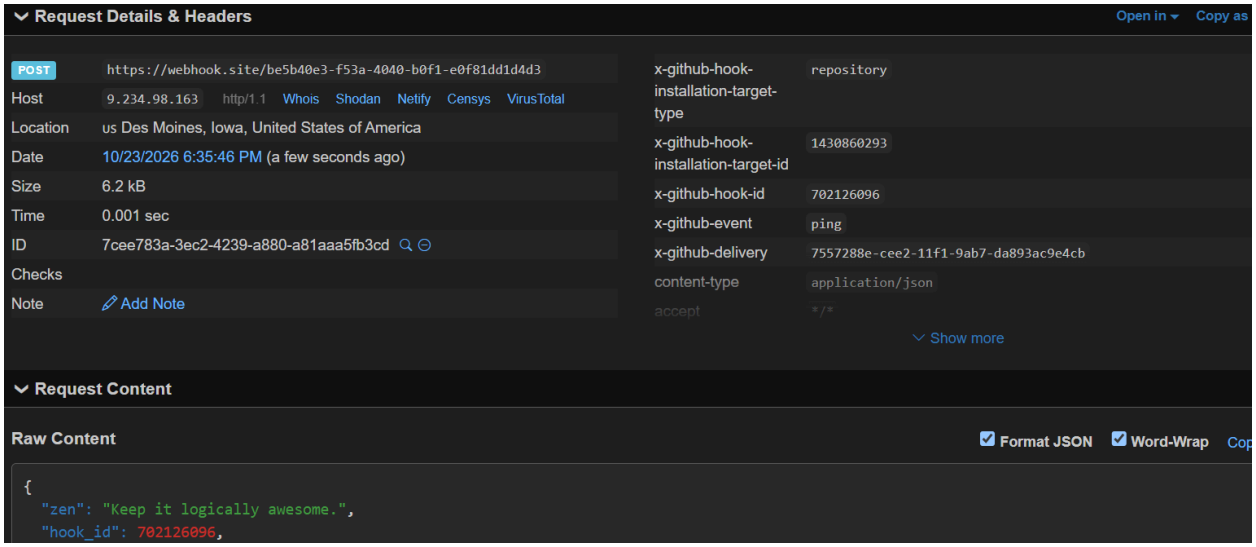
Step 3: Configure the Webhook

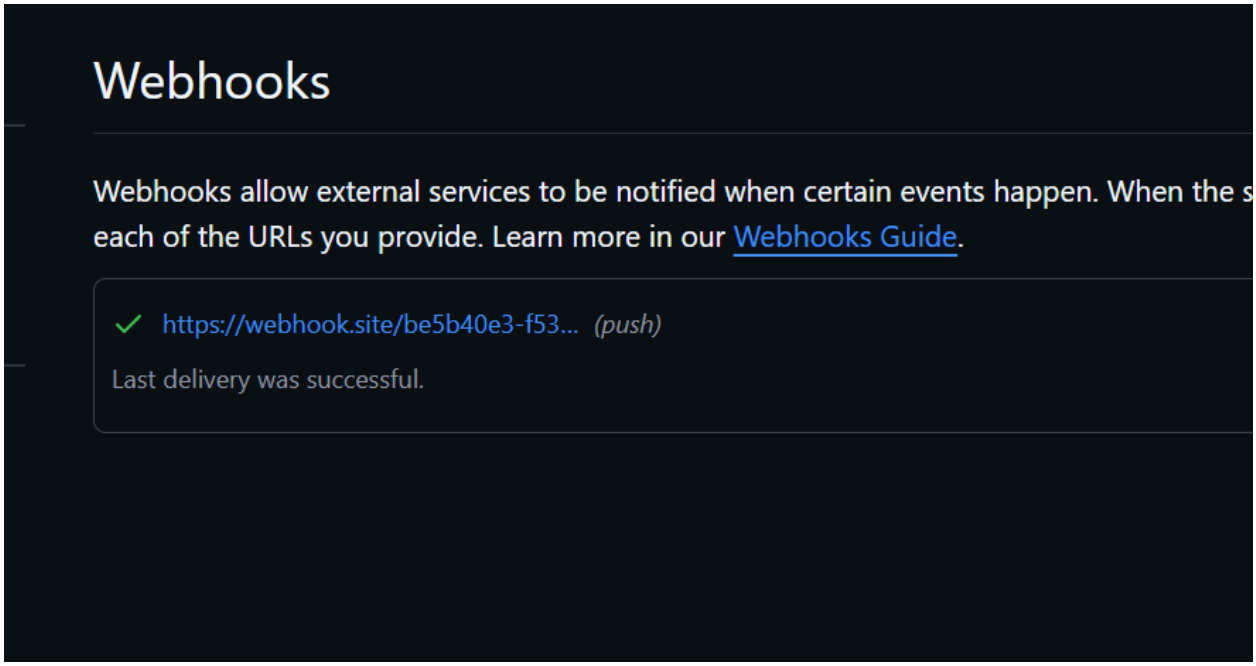
Enter the following details:

- **Payload URL:** `https://webhook.site/be5b40e3-f53a-4040-b0f1-e0f81dd1d4d3`
- **Content type:** `application/json`
- **Secret: Leave blank for this basic test.**
- **Which events would you like to trigger this webhook? Select Just the push event.**
- **Active: Keep it checked.**

Step 4: Save the Webhook

Click Add webhook to save the configuration.





19. Basic understanding of .git file.

The `.git` directory is a hidden folder created when you initialize or clone a Git repository. It stores all the important information Git needs to track changes and manage your project.

What does `.git` contain?

- **`objects` — Stores Git data, including file contents and commits.**
- **`refs` — Stores references to branches and tags.**
- **`HEAD` — Indicates the current branch or commit.**
- **`config` — Stores repository-specific configuration.**
- **`index` — Stores information about files staged for the next commit.**
- **`logs` — Records updates to references, such as branch changes.**

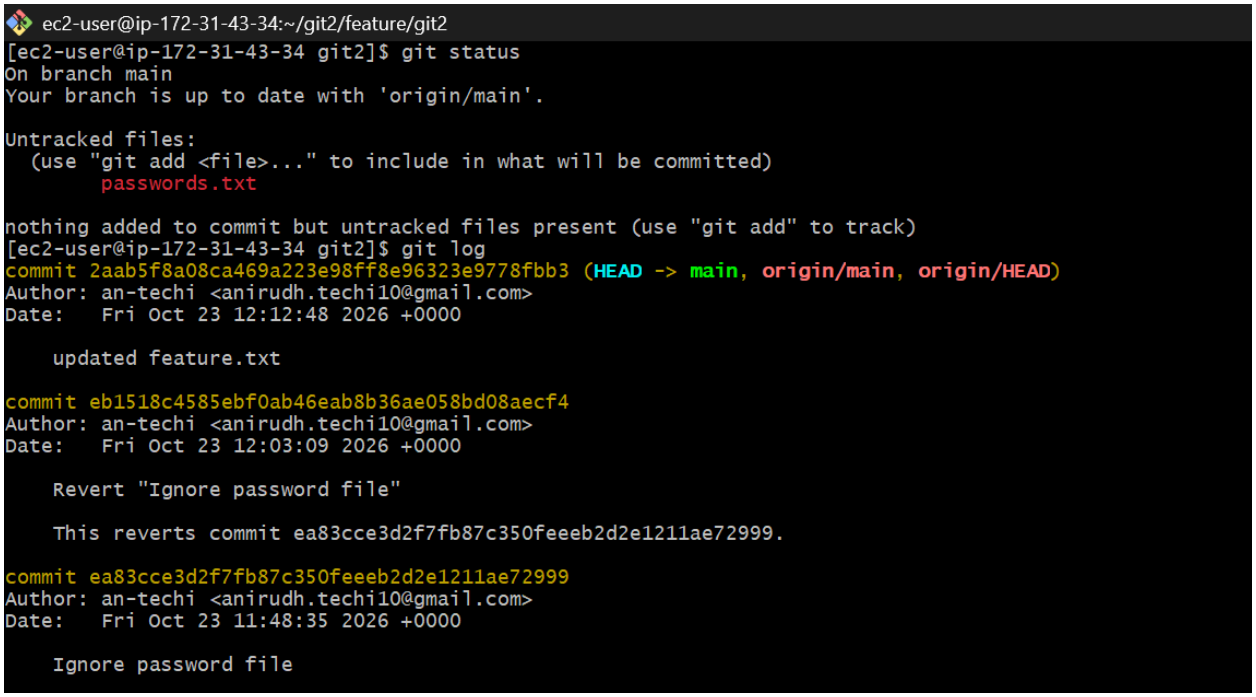
Important: `.git` is a directory, not a regular file. It manages your repository's history and internal data. If you delete it, your working files remain, but the repository's Git metadata and history are removed from that directory.

20. Check all the logs of git.

Command:

git log

Command explanation: Displays the commit history, including the commit ID, author, date, and commit message.



21. Rename the commit message.

Command:

git commit --amend

If the commit was already pushed to GitHub, changing its message changes the commit ID. You may need to push with `git push --force-with-lease` to update the remote branch. Use caution when working on shared branches.



22. Merge multiple commits into single commit.

Step 1: Start Interactive Rebase

git rebase -i HEAD~3

Command explanation:

- **`git rebase -i` — Opens interactive rebase to edit or combine commits.**
- **`HEAD~3` — Selects the last 3 commits.**

Step 2: Squash the Commits

An editor opens with a list similar to this:

pick abc123 First commit

pick def456 Second commit

pick ghi789 Third commit

Keep the first commit as `pick` and change the next two from `pick` to `squash` (or `s`):

pick abc123 First commit

squash def456 Second commit

squash ghi789 Third commit

Save and close the editor.

Step 3: Set the Final Commit Message

When prompted, enter one message for the combined commit, then save and close the editor.

Step 4: Verify the Result

git log --oneline

Command explanation: Displays the commit history so you can verify the commits were combined.

This rewrites commit history. If those commits were already pushed, coordinate with collaborators before updating the remote branch.

